GROUP16_LogiEdge_Final

> Measured results from a local end-to-end run on macOS (Apple Silicon) with Python 3.11, TensorFlow 2.15.1, Docker Compose, and Ansible. This simulation does not certify pharmaceutical safety.

Group: Group 16 — R. Ranjith N. (2025ae05379@wilp.bits-pilani.ac.in), P. Pricilla W. (2025ae05321@wilp.bits-pilani.ac.in), G. Guna Anban G. (2025ae05318@wilp.bits-pilani.ac.in)
Repository: https://github.com/rnagaraj13/LogiEdge (switch to private before deadline; add instructor + all members as collaborators)
Demo video: FILL_BEFORE_SUBMIT — tested unlisted/anyone-with-link URL

1 — FreightBridge Deployment Context (target 400–500 words)

FreightBridge's pilot monitors 85 refrigerated vehicles and may scale to 265. A refrigeration failure can raise cargo temperature by 1°C per minute, so LogiEdge must detect and alert within 90 seconds. That is an end-to-end sensor-to-local-alert SLA, not simply model invocation time. The specified rural routes have seven signal gaps lasting 35–90 minutes. A cloud-only system therefore cannot guarantee a timely decision or complete chain-of-custody record during those gaps. The truck must make classification and alarm decisions locally; cellular is used for eventual synchronisation and controlled model updates.

For a lower-bound bandwidth calculation, assume 32-bit sensor values and exclude timestamps and protocol framing. One 1 Hz temperature stream uses 4 bytes/s. Raw tri-axis vibration at 500 Hz uses `500 × 3 × 4 = 6,000 bytes/s`. The combined 6,004 bytes/s over a day is 518,745,600 bytes, or 518.75 decimal MB/truck-day. At ₹0.10/MB that is ₹51.87 per truck-day and about ₹4,409 per day for the 85-truck pilot. Real JSON/MQTT/TLS and retransmission overhead increases it. Edge summaries avoid transmitting every sample: an illustrative ten 1 KB alerts/day is 10 KB/day, approximately ₹0.001/truck-day, but alert frequency must be measured rather than assumed. The simulator's 0.5 Hz vibration RMS is an already aggregated value; the production sensor gateway must document the raw 500 Hz preprocessing.

The edge node buffers timestamped events, features, model versions and decisions in a durable local outbox. If cellular disappears, local MQTT, filtering, inference and alarms continue. When a secure uplink returns, a retrying sync service transfers deduplicated event IDs. On-device inference also limits access to detailed cargo readings, though confidentiality requires encryption, protected keys, least-privilege credentials, signed OTA and audited retention; edge processing alone does not prove contractual compliance.

The Constraint Triangle compares performance/latency, power/thermal envelope and cost. The selected pilot candidate is Raspberry Pi 5 (8 GB) plus AI HAT+ at the specified 7.5 W and ₹15,000 per truck. This is below the 10 W AI budget and costs ₹12.75 lakh for 85 trucks or ₹39.75 lakh for 265. Jetson Orin Nano Super is stated at 15 W and ₹45,000, so it exceeds power and costs ₹38.25 lakh pilot / ₹1.1925 crore at scale. STM32H7 costs ₹3,500 and 0.4 W but is not evidenced to support the selected

model/runtime, memory, OTA and SLA. Pi/HAT is conditional on measured sustained power, thermal behavior, sensor-to-alert latency, and runtime compatibility; 13 TOPS is not a timing result.

The supplied arithmetic intensity is `45 MFLOPs / 18 MB = 2.5 FLOP/byte`. The ridge point is `16 GFLOP/s / 12 GB/s = 1.33 FLOP/byte`, so the ideal Roofline classifies it as compute-bound. Peak lower bounds are 2.81 ms compute and 1.50 ms memory per inference; they are not measured latencies. The architecture is shown in Figure 1.

Measured lab evidence (this submission). Target production hardware remains Raspberry Pi 5 + AI HAT+ on paper; all timed runs in this report were executed on a macOS Apple Silicon laptop with Docker Desktop, not on the Pi. We therefore do not claim measured vehicle power or Pi thermal margins. On the laptop we completed one full offline `run_pipeline.py` experiment (dataset → train → PTQ → prune → benchmark → PSI reference → artifact staging) plus a live Mosquitto/inference stack demo. Held-out M2 mean inference latency over 200 timed calls was 0.0077 ms with p95 0.0082 ms; even adding the 30 s feature window and 10 s step, the designed sensor-to-alert path stays far below the 90-second SLA on this host. Offline cellular simulation was demonstrated by continuing local MQTT inference/alerts without an ops uplink; sync remains eventual. **SLA conclusion for this lab:** the software path meets the 90 s budget with large margin on the development host; Pi power/thermal/end-to-end confirmation is still required before a freight pilot go-live.

2 — Sensor Pipeline and MQTT Design (target 500–600 words)

Temperature publishes at 1 Hz; vibration arrives as 0.5 Hz RMS aggregated from the raw tri-axis 500 Hz signal; door contacts publish discrete OPEN/CLOSE events. The local Mosquitto tree is `logibridge/trucks/{truck_id}/sensor/{temperature|vibration_rms|door_event}`. The service publishes `.../inference` for every window and `.../alerts` for Warning, Critical and door-open events. Temperature/vibration use QoS 0 because each periodic value is superseded, while door events and decisions use QoS 1 for at-least-once delivery. The sync agent deduplicates using content-hash event IDs; alert and outbox persistence are local. No topic is retained. Retained `health` and `ota/ack` topics are planned but not implemented. The complete tree and justifications are in `data_pipeline/mqtt_architecture.md`.

The preprocessing path applies a five-sample moving average separately to temperature and vibration. A 30-second window advances every ten seconds. Temperature produces mean, standard deviation and rate in °C/min; vibration produces RMS, peak and Pearson kurtosis. These independently computed features are concatenated as a stable six-value vector. Feature-level fusion is appropriate because it captures joint sensor relations without requiring an MLP to consume the raw high-rate waveform. Data-level fusion would require time-aligning streams with very different rates and would transport/process far more values. Decision-level fusion would require independent classifiers and a later rule/model to combine outputs, obscuring cross-sensor relations. Door events remain logged and alerted separately because the assigned model input has exactly six values; they are not silently represented as an unrequested seventh feature.

The training normalization parameters are calculated from ten minutes of clean Normal simulation and stored in `training_stats.npy`. Inference loads that frozen file; it never learns statistics from live data, because adapting to a drift could hide the drift itself. A required sensitivity test compares the held-out model accuracy under correct statistics and statistics whose mean is shifted by 3σ while preserving the

original scale. The problem also has a label inconsistency: +0.08°C per second crosses the Critical >3°C deviation after 38 seconds, although the dataset table assigns an entire 15-minute `temp_drift` run to Warning. This implementation preserves the specified ramp and labels windows by severity, treats vibration-only bearing anomaly as Warning, and treats combined as Critical. The resulting mode policy is recorded in the generation manifest and should be confirmed with the instructor.

Normalization case	Held-out accuracy	Change from correct stats
Correct ten-minute clean mean/std	100.00%	Baseline
Mean shifted by 3σ	94.74%	–5.26 percentage points

Measured notes. Seed 631; stratified 20% hold-out gave 302 train / 76 validation windows from 378 total labeled windows (class counts Normal 117, Warning 88, Critical 173 per `generation_manifest.json`). With correct frozen stats, validation accuracy was 100.00%. After shifting only the normalization mean by +3σ (std unchanged), accuracy fell to 94.74% (–5.26 pp) on the same split (`normalization_shift_experiment.json`). The drop shows the classifier is sensitive to the locked Normal baseline: if live systems silently refit mean/std during a fault, severity can be under-stated. Keeping `training_stats.npy` immutable at inference is therefore a safety control, not a convenience.

3 — Model Deployment and Pipeline Mapping (target 400–500 words)

The training script generates the Normal, Warning and Critical windows, freezes clean-data statistics, trains the two-hidden-layer 32/16 ReLU MLP, and uses a stratified 20% held-out split. It records accuracy, confusion matrix and class recall, and stops if accuracy is below the required 88%. The deployment candidate must additionally exceed 95% Class 2 recall. PTQ conversion uses at least 200 representative windows and full integer TFLite input/output. The third candidate uses a TensorFlow Model Optimization Toolkit PolynomialDecay schedule to remove whole hidden units, compacts their dimensions, then performs full INT8 PTQ; report measured size and accuracy to verify the optimization did not trade away safety-critical recall.

The ten lifecycle stages are mapped specifically in `reports/pipeline_mapping.md`: acquisition, validation, feature engineering, frozen normalization, training, validation/selection, conversion/optimization, packaging, edge deployment/inference, and monitoring/update. The assignment statement does not list the official ten stage names; compare the table labels with the exact Module 4 lecture terms before final export.

The Docker image uses Python 3.11 slim and installs pip dependencies before `COPY model.tflite .`. This preserves dependency layers when the model alone changes. `MODEL_PATH` selects a mounted TFLite variant without rebuilding. The inference service subscribes locally, executes preprocessing and model inference, writes persistent local events, and publishes decisions without the cellular uplink. The sync agent keeps a SQLite outbox and forwards events over configured TLS when coverage returns.

The OTA nominal calculation treats the supplied 280 KB model as 0.28 decimal MB. Full fleet transfer for 85 trucks is 23.8 MB and ₹2.38. A ten-truck canary costs ₹0.28 for its 2.8 MB initial phase; the remaining 75 trucks cost ₹2.10, for the same ₹2.38 after complete rollout. Shadow mode also transfers the candidate and adds unquantified compute/telemetry. A staged canary is preferred for blast-radius control and rural retry behavior, not for reducing total completed transfer bytes.

Measured Docker / OTA bytes. Inspecting `logibridge-inference:latest` on the lab host: total image size 219,492,906 bytes (~209.3 MiB). `docker history` shows the dependency install layer ≈69.2 MB and the isolated `COPY model.tflite` layer = 3,544 bytes (matches `training/models/m2_int8.tflite`). Stats file layer is 176 bytes. Because dependencies are installed before the model copy, a model-only rebuild replaces the 3.5 KB layer rather than re-transferring the ~69 MB pip layer. Versus shipping a full 219 MB image to 85 trucks (≈18.7 GB), a 3,544-byte model-only update is ≈301 KB fleet-wide—orders of magnitude cheaper than full-image OTA. Using the assignment's ₹0.10/MB SIM tariff on the measured 3,544-byte model gives a negligible per-truck cost compared with ₹2.38 under the 280 KB nominal assumption; we still quote the assignment's ₹2.38 canary arithmetic for comparability, while noting our actual INT8 artifact is much smaller than 280 KB.

4 — Optimisation and Pareto Analysis (target 400–500 words)

The benchmarking script performs ten excluded warm-ups and 200 measured calls per variant. It reports mean inference latency, p95 latency, model size, held-out accuracy and energy/inference. The energy estimate uses `psutil` process CPU percentage and an explicitly configured laptop TDP estimate; it is a comparative estimate, not a calibrated edge-board power measurement. M1 is the FP32 Keras baseline, M2 uses full INT8 post-training quantization and M3 combines structured hidden-unit pruning with PTQ INT8. The Pareto chart uses measured latency and size for visualization; non-dominance is calculated across latency, size, energy and accuracy. An attractive size/latency point is not eligible if its Critical recall fails the stated threshold.

Variant	Mean latency ms (200)	p95 ms	Size KB	Accuracy %	Energy mJ/inference
M1 FP32 baseline	0.729	0.460	35.934	100.00	1.821
M2 full INT8 PTQ	0.00772	0.00817	3.544	100.00	0.0192
M3 structured prune + PTQ INT8	0.00771	0.00788	3.128	98.68	0.0193

Class 2 recall is separately evaluated from `recall = true critical predictions / all actual critical validation windows`. Record each numerator and denominator from the confusion matrix, not a recollection. **M2 Critical recall = 100.00% (35/35 held-out Critical windows; confusion matrix diagonal for class 2).** This is strictly above the 95% gate. M1 also scored 35/35. M3 scored 97.14% Critical recall (34/35) with compacted hidden widths 21 and 11 after 35% structured sparsity—still above 95%, but slightly weaker than M2 on this split.

The 90-second SLA includes the 30-second feature window and ten-second step. A newly appearing fault may wait for the next window step; the implementation must measure the actual worst-case sensor-to-alert path and demonstrate it remains below 90 seconds. The Pi/HAT option fits the stated 10 W budget on paper, but actual memory/flash/RSS, sustained power, runtime and accelerator usage need target-device measurement. TOPS and ideal Roofline bounds are not evidence of deployment speed. My recommendation is **M2 full INT8 PTQ**: it meets ≥88% accuracy (100%) and >95% Critical

recall (100%), cuts size from 35.9 KB to 3.54 KB versus M1, and reduces mean latency by roughly two orders of magnitude on this host. M3 is a close runner-up for storage, but we prefer M2's perfect Critical recall on the measured hold-out. Energy values used `LOGIBRIDGE_LAPTOP_TDP_WATTS=25` and are comparative only. Include Pareto chart generated by `optimisation/benchmark.py`.

5 — MLOps Monitoring and Reflection (target 300–400 words)

The clean confidence reference is generated by running the selected model on 300 independent Normal simulation windows. Maximum class confidence is binned as [0,.25), [.25,.5), [.5,.75), and [.75,1]. PSI compares the reference proportions to the most recent 100 inferences every 60 seconds with a small epsilon for empty bins. PSI above 0.25 logs a drift alert; after restoring clean data, a result below 0.10 indicates recovery. The injected mode is a covariate/operational distribution shift in simulated inputs; confidence-only PSI detects a change in output confidence distribution, not the root cause or correctness of labels.

Phase	PSI / time / sample evidence
Clean baseline	Reference `m2_int8.tflite`, n=300, proportions [0.00, 0.857, 0.00, 0.143]; mean confidence 0.427
Combined drift injection (60–1140 s)	First PSI after n=100 (~17 min): **13.367** → DRIFT ALERT, bins [0, 0, 0, 1.0]; 16 consecutive alerts while faulty windows remained in the buffer. Log: `psi_recovery_evidence.txt`
Restored clean input (from 1140 s)	PSI fell every minute (1.94 → 0.88 → 0.41 → 0.29 → 0.17 → 0.10); **DRIFT RECOVERED at PSI 0.031** about 16 min after clean data resumed, then settled at 0.000–0.010 with bins [0, 0.85, 0, 0.15]

The Ansible play has exactly seven tasks and is designed to avoid stopping an unchanged container. Run it twice with immutable image/artifact inputs.

PLAY RECAP run 1: `localhost : ok=6 changed=1 unreachable=0 failed=0 skipped=1`
PLAY RECAP run 2: `localhost : ok=6 changed=0 unreachable=0 failed=0 skipped=1`

Second run confirms idempotency (`changed=0`). Artifacts logged in `ansible_run1.log` / `ansible_run2.log`. Local adaptations: `ansible_become=false`, user-writable `logibridge_dir`, absolute model/reference paths, and `pull: false` so the already-built local image is used without Docker Hub. The sync agent queues events locally and replays them over authenticated TLS when uplink returns; locally generated alerts do not wait on that path.

Genuine technical difficulty: On Apple Silicon, `tensorflow-cpu==2.15.1` has no wheel, so we installed `tensorflow==2.15.1` plus `tf_keras==2.15.1`. Structured pruning then failed because TFMOT's `PolynomialDecay` returns `(should_update, sparsity)` rather than a single tensor; `prune_quantise.py` was updated to unpack the tuple. Separately, PSI needs ~100 inferences before the first print (~15–20 min at a 10 s step), which initially looked like a "silent" monitor until we understood the sample gate. Ansible `copy` also resolved relative paths from `deployment/`, not the repo root—fixed with absolute `model_source` / `reference_source`.

One architectural change: Rebuild the PSI reference from live clean MQTT inferences of the deployed container (same preprocessing path) instead of only offline TFLite windows, and/or add a feature-space PSI alongside confidence PSI. An earlier run never reached PSI < 0.10; the likely cause was a second inference container sharing the same MQTT client ID, so the two kept disconnecting each other. With one container, recovery was measured, but the ~16-minute recovery lag shows a 100-sample buffer is slow; a time-decayed window would confirm recovery sooner.

The outputs are synthetic training/engineering evidence. A successful lab run is not certification for pharmaceutical deployment; a real fleet pilot requires qualified sensors, traceable calibration, hardware/environment tests, security review, operating procedures, and clinical/customer approval.

Conclusion

We generated a reproducible offline experiment and live edge stack. Held-out M1/M2 accuracy reached 100% with Critical recall 35/35; M3 retained 98.68% accuracy and 97.14% Critical recall after prune+PTQ. Benchmarking favors **M2** for deployment among eligible variants. Live PSI exceeded 0.25 (peak 13.37) under combined stress and returned below 0.10 (DRIFT RECOVERED at 0.031) once clean data refilled the 100-sample window. Ansible demonstrated seven-task rollout with second-run `changed=0`. **Conditional go for a simulated lab pilot / course demo; no-go for unsupervised pharmaceutical fleet deployment** until Pi power/thermal SLA tests, security hardening, and real-sensor validation are completed.

References (excluded from word count)

1. FreightBridge LogiEdge Mini-Project Problem Statement — Student, supplied course document.
2. TensorFlow Lite documentation: https://www.tensorflow.org/lite
3. Mosquitto documentation: https://mosquitto.org/documentation/
4. Ansible documentation: https://docs.ansible.com/
5. TensorFlow Model Optimization documentation: https://www.tensorflow.org/model_optimization
6. Local evidence files: `training/results/training_metrics.json`, `training/results/normalization_shift_experiment.json`, `optimisation/results/benchmark_results.csv`, `monitoring/reference_dist.json`, `reports/evidence/psi_recovery_evidence.txt`, `reports/evidence/inference_run_2026-11-05.txt`, `ansible_run1.log`, `ansible_run2.log`.